HW4: midterm practice

Joshua Wilson

10-1-26

Questions generated using chatgpt, below you can see the shared link to the chat gpt chat:

https://chatgpt.com/share/6abff60b-be10-83ea-9647-4ef673359675

Question 1 — Deep Neural Network Architecture and Mechanics

Consider a fully connected feed-forward neural network with an input layer, two hidden layers, and an output layer. The hidden layers use nonlinear activation functions.

Explain, both conceptually and mathematically, how such a network transforms an input into a prediction. Your answer should address:

- The role of **weights, biases, and activation functions** in each layer.
- Why stacking multiple layers with **linear transformations alone** does not increase the expressive power of the network.
- How nonlinear activations allow a deep network to represent complex functions.
- The **forward-pass equations** for a two-hidden-layer network.
- The intuition behind the **universal approximation theorem** and why it does *not* imply that a shallow network is necessarily preferable to a deep one.
- How the **chain rule and backpropagation** determine how each parameter is updated during training.
- Explain the **vanishing-gradient problem** and why activation-function choice can affect it.

Question 2 — Loss Functions and Optimization

A neural network produces predictions $\hat{y}$from input data x, and its parameters $\theta$are learned by minimizing a loss function.

Explain the relationship between the loss function, gradients, and optimization in neural-network training. Your answer should include:

- The distinction between a **loss function for an individual example** and an overall **training objective** such as empirical risk.
- Derive the gradient-descent update

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t)$$

and explain mathematically what the learning rate ηcontrols.

- Compare **mean squared error** and **cross-entropy loss**, explaining why different losses are appropriate for different prediction problems.
- For binary classification, explain why the combination of a **sigmoid output and binary cross-entropy** is mathematically natural.
- Explain the intuition behind **stochastic gradient descent (SGD)** and why using mini-batches can be beneficial.
- Discuss how optimization can be affected by **learning rate, local curvature, saddle points, and poorly conditioned loss landscapes**.
- Explain conceptually how methods such as **momentum or adaptive learning rates** attempt to improve optimization.

Finally, distinguish between **minimizing training loss** and obtaining good **generalization performance**. Why can an optimization procedure successfully minimize training loss while the resulting model performs poorly on unseen data?

Question 3 — Convolutional Neural Networks

Suppose you are given a color image of dimensions $32 \times 32 \times 3$, and you want to use a neural network to classify the image.

Explain why a Convolutional Neural Network (CNN) can be more appropriate than a fully connected network for this task. Your answer should develop the argument from both architectural and mathematical perspectives.

Address the following:

- Explain the operation of a **2D convolution** and describe the roles of the **kernel/filter, stride, padding, and receptive field**.
- Given an input of spatial size 32×32, a 5×5filter, stride 1, and no padding, derive the spatial dimensions of the resulting feature map.
- Explain why a convolutional layer has **parameter sharing** and how this reduces the number of parameters compared with a fully connected layer.
- Explain how convolution gives CNNs an inductive bias toward **local spatial structure**.

- Describe how multiple convolutional layers cause the network's **receptive fields** to grow and how this can lead to hierarchical feature representations.
- Explain the roles of **pooling and/or strided convolutions**, including their effects on spatial resolution and computational cost.
- Discuss why CNNs are often relatively robust to small translations of objects in images, while also explaining why ordinary convolution is **not inherently fully translation-invariant**.
- Explain how the output of convolutional layers can ultimately be transformed into class predictions.

Conclude by comparing the **representational assumptions** made by a CNN with those made by a fully connected neural network when processing images.

Unedited answers:

Question 1

A neural network is a network that makes a prediction about a given input, the prediction can be an integer or a string. The network in plain terms is a system of equations put together in order to traverse the network which consist of neurons or links. The system of equations are made up of weights which influence hidden units making the unit go up or down, there is a bias for every hidden unit which influences direction, and the activation is the most important which ensures our system of equations are non-linear as well as ensuring our output is within a given range

Stacking multiple layers with linear transformation alone does not increase the expressive power of the network because, linear outputs are constant and predictable, while non-linear transformations do the opposite and give us variation. In addition to this non-linear activations allow deep networks to represent complex functions because complex functions are typically not linear and a non-linear activation forces our system of equations to be nonlinear.

Forward pass equation is the portion of our system of equations which moves from layer to layer(best guess).

The universal approximation theorem builds off of our topic of variation in outputs that an increase in neurons with a decrease in layers doesn’t improve our network, at times it may make it worse or slower.

The chain rule is used for our loss functions, it is a basic calculus principle that helps us that the derivative of an exponential equation. This rule is used for our loss function, which is a type of function that determines the loss of data in our network. It essentially helps us test our network. Depending on if the loss (output of the loss function) is high or low we will most likely want to modify our parameters if loss is high because it means we are lossing data somewhere in our network and making our predictions less accurate because of it.

Vanishing gradient problem is an issue where we lose information due to our choice of activation function (best guess).

However depth is important in a neural network because it affects the accuracy of our prediction.

Question 2:

The distinction between a loss function for an individual example and an overall training objective such as empirical risk, is that the loss function is for updating parameters and training objective is for updating training data (best guess).

No idea how to derive the gradient-descent update, best guess is that you would actually take the derivative.

Mean square is for large outputs cross entropy is for overlapping outputs(best guess).

No idea why sigmoid would be combined with binary cross entropy, best guess is to get a more accurate idea where loss is occurring.

No idea what stochastic gradient descent is, best guess is it is used in mini batches to get more accurate loss output.

Optimization can be affected by learning rate because lower learning rate means our network isn't gradually making better predictions. Local curvature I think is in reference to our graph of hidden units, less curve means less accuracy. I do not know what a saddle point is.

Adaptive learning rates attempts to improve optimization by learning more as time goes on.

Question3:

A CNN would definitely be more suited to handle a three dimensional input image. The 2d convolution of a cnn gives us a separate type of information along the way. It gives us the kernel size which is the max number of inputs per neuron in a network, we have stride which is the distance between inputs and padding which is the distance between outputs and receptive field I'm not sure.

32 x 5 would be the dimensions of the feature map

Convolutional layers have parameter sharing because the next layer depends on the previous, this reduces the number of parameters

I do not know what local spatial structure is, my best guess is that cnn's give inductive bias because each layer affects the next.

More layers increase the receptive field to grow because we are able to more calculations there fore increasing the input will not affect the output.

I have no idea but my guess would be strided convolutions are used to help boost accuracy in our system of equations.

Cnn's are often robust to small translations of objects in images because they are built to optimize images while making predictions.

Convolution layers are used to detect patterns and edges of objects. Each layer determines a portion of the image; if that portion is an arch, or a line, or even an object. It determines these things at each layer which influence what the image classification will be.

edited answers Note edited answers come partially or entirely from chat gpt and are marked by bold

1: Question 1

A neural network is a network that makes a prediction about a given input, the prediction can be an integer or a string. The network in plain terms is a system of equations put together in order to traverse the network which consist of neurons or links. The system of equations are made up of weights which **influence how strongly different input features contribute to the neurons pre-activation**, there is a bias for every hidden unit which influences direction, and the activation is the most important which ensures our system of equations are non-linear.

Stacking multiple layers with linear transformation alone does not increase the expressive power of the network because, **a composition of linear transformations is itself a linear transformation**.

In addition to this non-linear activations allow deep networks to represent complex functions because complex functions are typically not linear and a non-linear activation forces our system of equations to be nonlinear. The activation function σprevents the entire network from collapsing into one linear transformation.

This allows the network to represent functions involving:

- nonlinear decision boundaries

- interactions between features
- hierarchical representations
- complicated mappings from input to output

Forward pass equation is the portion of our system of equations which moves from layer to layer **This is the forward pass.**

Think:

Linear transformation → activation → linear transformation → activation → ... → prediction

The universal approximation theorem is a sufficiently wide neural network with an appropriate nonlinear activation function can approximate a very broad class of continuous functions to arbitrary accuracy, given enough hidden units. it tells us that a sufficiently wide network **can theoretically approximate** complex functions. But depth can make certain functions much more **parameter-efficient** or easier to represent. This is an important distinction: **Universal approximation is about existence of an approximation, not necessarily efficiency, learnability, or practical performance.**

The chain rule is used to calculate **how the loss changes with respect to parameters throughout the network .** High loss means t**he model's prediction is poorly aligned with the target according to the chosen loss function.**

Vanishing gradient problem is an issue where we lose information due to our choice of activation function (best guess). **Earlier layers receive extremely small updates and learn very slowly.** This is the **vanishing-gradient problem**.

Depth allows a network to compose multiple nonlinear transformations, enabling hierarchical representations and potentially representing certain complex functions more efficiently than a shallow network.

Question 2:

The distinction between a loss function for an individual example and an overall training objective such as empirical risk, is This is the **empirical risk** or average training loss. So: **Individual loss = performance on one example. Training objective = aggregate loss across the training dataset that we optimize with respect to model parameters.**

The way gradient-descent update is derived, is actually take the derivative.

Suppose:

$$J(\theta)$$

is our objective.

The gradient is:

$$\nabla_\theta J(\theta)$$

It tells us the direction in parameter space where the objective increases most rapidly.

Therefore, to reduce the objective, move in the opposite direction:

$$\boxed{\theta_{t+1} = \theta_t - \eta \nabla_\theta J(\theta_t)}$$

where $\eta$is the learning rate. The learning rate controls the **step size**. **Too small:** Training can be extremely slow. **Too large:** You can:

- overshoot minima
- oscillate
- become unstable
- fail to converge

MSE is commonly associated with **regression** problems. Cross-entropy is commonly used for **classification**, especially probabilistic classification, for binary classification.

MSE → commonly regression

Cross-entropy → commonly classification

Sigmoid and binary cross entropy are combined so that The deeper answer is **probabilistic modeling**. **Sigmoid + binary cross-entropy corresponds naturally to the Bernoulli likelihood and produces a particularly convenient gradient.**

Suppose your dataset contains:

$$1{,}000{,}000$$

training examples.

Computing the exact gradient over all million examples for every parameter update would be expensive.

Instead, use a **mini-batch**, perhaps 32 or 128 examples.

Compute an approximate gradient:

$$\nabla_\theta J_{\text{batch}}$$

and update:

$$\theta \leftarrow \theta - \eta \nabla_\theta J_{\text{batch}}$$

That's **mini-batch stochastic gradient descent**.

The gradient is noisy because it's based on only a subset of the data.

That noise isn't necessarily bad; it can sometimes help optimization move through complicated loss landscapes. For example, one direction might be extremely steep while another is very flat.

Then a single learning rate may be:

- too large for the steep direction
- too small for the flat direction

Curvature describes **how rapidly the slope changes**.

A sharply curved direction might look like:

U

while a relatively flat direction looks more like:

—

Optimization becomes difficult when different directions have dramatically different curvature.

This is related to **ill-conditioning**.

A saddle point is a point that is neither minimum or maximum along the gradient.

Adaptive learning**: Different parameters can receive different effective learning rates based on information about their historical gradients. For example, Adam maintains estimates related to the first and second moments of gradients and uses them to adapt updates. You don't necessarily need to memorize the entire Adam derivation unless your course specifically requires it, but you should understand the motivation.**

Question3:

Kernel/Filter: A small matrix of learnable weights sliding over input spatial locations to detect features (e.g., edges, textures).

Stride (s): The step size (in pixels) the filter moves across the image during scanning.

Padding (p): Adding outer border pixels (usually zeros) to control the output map's spatial boundary dimensions.

Receptive Field: The spatial region of the input image that directly influences the activation of a specific downstream neuron.

28x28 would be the dimensions of the feature map

$$H_{out} = \frac{32 - 5}{1} + 1 = 28, \quad W_{out} = \frac{32 - 5}{1} + 1 = 28$$

In fully connected (FC) layers, every output node connects to every input pixel independently. For a $32 \times 32 \times 3$ image flattened to 3072 inputs connecting to 1000 units, an FC layer requires $3{,}072 \times 1{,}000 = 3{,}072{,}000$ parameters. In a CNN, the **same kernel weights are shared and swept across all spatial locations**. A $5 \times 5 \times 3$ filter applied across the image uses only $5 \times 5 \times 3 + 1 = 76$ parameters regardless of input image size, drastically reducing model footprint and overfitting risk.

CNNs enforce two structural domain assumptions: **Locality:** Nearby pixels are strongly correlated; distant pixels are less so. Filters process small contiguous patches at a time. **Translation Equivariance:** A feature (like an edge or eye) retains its meaning regardless of where it appears in the image. Passing the same filter across all locations applies the same visual check universally.

Stacking convolutional layers expands the effective receptive field compoundingly: **Layer 1:** 3×3 filter looks at a 3×3 input region (detects simple low-level edges). **Layer 2:** A 3×3 filter applied over Layer 1's outputs effectively looks at a 5×5 region of the original input image (detects textures/corners). **Deeper Layers:** Combine low-level features over wider receptive

fields to represent high-level semantic concepts (e.g., wheels, faces, objects). **Pooling (Max/Average) & Strided Convolutions:** Downsample feature map spatial dimensions (H, W). **Effects:** Reduces memory consumption and computational cost for deeper layers. Increases the receptive field growth rate per subsequent layer. Provides spatial invariance to minor distortions.

Convolutional Layers are Equivariant, NOT Invariant: If input X shifts by t, feature map $f(X)$ shifts by the same amount t: $f\big(g_t(X)\big) = g_t\big(f(X)\big)$. **How Invariance is Achieved:** Global pooling operations (e.g., Max Pooling or Global Average Pooling) strip away absolute spatial coordinates, allowing the network to output identical class predictions regardless of exact feature positions within the frame.

Convolutional & Pooling Layers: Extract spatial feature maps ($H' \times W' \times C$). **Global Average Pooling / Flattening:** Collapses spatial dimensions ($H' \times W'$) into a single feature vector of length C. **Fully Connected Layer:** Maps feature vector to K output class logits. **Softmax Activation:** Normalizes logits into normalized class probability distribution predictions:

$$P(y = k|x) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

What have I learned? I have learned that generative AI is very useful for getting a deeper understanding of definitions. I learned a little bit more about CNN's and plan to use AI for studying in the future. My advice to others using gen AI is to be as specific as possible when entering a prompt but always ask questions that you don't know the answer to.